Algorithmes pour l'optimisation en nombres entiers

Optimisation Continue Sous Contraintes

Relaxation lagrangienne et programmation dynamique

AFTISSE Melissa et RAVENDIRANE Gayathiri
Master MAS parcours ROAD,
Année universitaire : 2025−2026

Table des matières

1 Problèmes et relaxations **3**
1.1 Instances pour le développement et débogage 3
1.2 Problème d'ordonnancement 3

2 Méthodes de sous-gradient implémentées **4**
2.1 Méthode de sous-gradient basique 4
2.2 Méthode de sous-gradient avec règle de Polyak 5
2.3 Méthode ADS (Average Direction Strategy) 5
2.4 Méthode des plans coupants 6
2.5 Tableau comparatif des méthodes 6

3 Programmation dynamique **7**
3.1 Relaxation 1 : relaxation des contraintes (6) 7
3.2 Relaxation 2 : relaxation surrogate des contraintes (6) 7
3.3 Reconstruction de la solution et vérification de sa faisabilité 8

4 Expérimentations et analyse des résultats **8**

5 Conclusion **10**

6 Annexe **11**

1 Problèmes et relaxations

1.1 Instances pour le développement et débogage

Nous définissons les deux problèmes suivants :

$$(P_1) : \begin{cases} \min & f(x) = x_1 - 4x_2 \\ \text{s.c.} & -x_1 - x_2 + 2 \leq 0 \\ & x_2 - 1 \leq 0 \\ & x \in \mathcal{X} := \{x : 0 \leq x_1 \leq 3, 0 \leq x_2 \leq 3\} \end{cases}$$

$$(P_2) : \begin{cases} \min & f(x) = (x_1 - 2)^2 + \frac{1}{4}x_2^2 \\ \text{s.c.} & x_1 - \frac{7}{2}x_2 - 1 \leq 0 \\ & x \in \mathcal{X} := \{x : 2x_1 + 3x_2 = 4\} \end{cases}$$

1.2 Problème d'ordonnancement

Le problème suivant est un problème d'ordonnancement de tâches sur une seule machine afin de minimiser la somme des retards pondérés noté $1||\sum w_i T_i$. Nous considérons un ensemble de tâches noté I ou chaque tâche $i \in I$ possède une durée de traitement donnée par p_i. Nous définissons également l'ensemble T qui représente l'horizon temporel global avec $T = \{0, \cdots \sum_{i \in I} p_i\}$. Pour chaque tâche $i \in I$ nous notons l'ensemble des dates de début possibles par $T_i = \{0, \ldots T - p_i\}$. L'objectif (1) est de minimiser la somme des coûts de tardivité définis par $c_{it} = w_i \times max\{0, t + p_i - d_i\}$ avec w_i le poids de la tâche i et d_i sa date d'echéance.
Nous introduisons notre variable x_{it} qui vaut 1 si la tâche i a commencé à l'instant t ($\forall i \in I$ et $\forall t \in T_i$).

$$\min \quad \sum_{i \in I} \sum_{t \in T_i} c_{it} x_{it} \tag{1}$$

$$\text{s.t.} \quad \sum_{t \in T_i} x_{it} = 1, \quad \forall i \in I \tag{2}$$

$$\sum_{i \in I} \sum_{t' \in \{t - p_i + 1, \ldots, t\} \cap T_i} x_{it'} \leq 1, \quad \forall t \in T \tag{3}$$

$$x_{it} \in \{0, 1\}, \quad \forall i \in I, t \in T_i \tag{4}$$

Les contraintes (2) imposent l'unicité d'une date de début pour une tâche i tandis que les contraintes (3) assurent le non chevauchement des tâches sur la machine.

Dans ce qui suit, nous noterons (P_3) le problème dual lagrangien issu de la relaxation des contraintes (3). De même (P_4) est issu en relachant les contraintes (2), ajoutant la

contrainte $\sum_{i\in I}\sum_{t\in T_i} x_{it} = n$ et transformant les contraintes (3) en égalités.

$$
\begin{aligned}
(P_3) : \max_{\mu\in\mathbb{R}^n_+} \quad & L(\mu) = \max_{\mu\in\mathbb{R}^n_+}\min_{x\in X} \mathcal{L}(x,\mu) \\
& = \max_{\mu\in\mathbb{R}^n_+}\min_{x\in X} \sum_{i\in I}\sum_{t\in T_i}(c_{it} + \sum_{s=t}^{t+p_i+1}\mu_s)x_{it} - \sum_{t\in T}\mu_t \\
& = \max_{\mu\in\mathbb{R}^n_+} k \\
\text{s.t.} \quad & k \leq \sum_{i\in I}\sum_{t\in T_i}(c_{it} + \sum_{s=t}^{t+p_i+1}\mu_s)x_{it} - \sum_{t\in T}\mu_t \\
& \sum_{i\in I}\sum_{t'\in\{t-p_i+1,\dots,t\}\cap T_i} x_{it'} = 1, \quad \forall t\in T \\
& \sum_{i\in I}\sum_{t\in T_i} x_{it} = n \\
& x_{it}\in\{0,1\}, \quad \forall i\in I, t\in T_i \\
& \mu\in\mathbb{R}^n_+, \quad k\in\mathbb{R}
\end{aligned}
$$

$$
\begin{aligned}
(P_4) : \max_{\mu\in\mathbb{R}^n} \quad & L(\mu) = \max_{\mu\in\mathbb{R}^n}\min_{x\in X} \mathcal{L}(x,\mu) \\
& = \max_{\mu\in\mathbb{R}^n}\min_{x\in X} \sum_{i\in I}\sum_{t\in T_i}(c_{it} + \mu_i)x_{it} - \sum_{i\in I}\mu_i \\
& = \max_{\mu\in\mathbb{R}^n} k \\
\text{s.t.} \quad & k \leq \sum_{i\in I}\sum_{t\in T_i}(c_{it} + \mu_i)x_{it} - \sum_{i\in I}\mu_i \\
& \sum_{i\in I}\sum_{t'\in\{t-p_i+1,\dots,t\}\cap T_i} x_{it'} = 1, \quad \forall t\in T \\
& \sum_{i\in I}\sum_{t\in T_i} x_{it} = n \\
& x_{it}\in\{0,1\}, \quad \forall i\in I, t\in T_i \\
& \mu\in\mathbb{R}^n, \quad k\in\mathbb{R}
\end{aligned}
$$

2 Méthodes de sous-gradient implémentées

2.1 Méthode de sous-gradient basique

L'une des méthodes les plus simples pour résoudre le problème dual lagrangien. À chaque itération, on calcule un sous-gradient puis on met à jour les multiplicateurs λ^k avec un pas s_k.

Principe : Avec cette méthode nous générons une suite d'itérés $(\lambda^k)_{k\in\mathbb{N}}$ par une règle de mise à jour itérative. À chaque étape k, les multiplicateurs sont mis à jour comme suit :

$$\lambda^{k+1} = P_\Omega(\lambda^k + s_k d^k)$$

où $d^k \in \partial L(\lambda^k)$ représente une direction de déplacement dans le sous-différentiel de L au point λ^k, $s_k > 0$: le pas de l'itération et P_Ω : l'opérateur de projection sur l'espace des contraintes admissibles Ω assurant la réalisablité des itérés.

Limites : Le sous-gradient peut être instable et ne pas orienter la recherche de manière efficace vers une direction d'amélioration, ce qui peut entraîner des oscillations importantes des itérés (effet de zigzag). Par conséquent, la convergence est souvent lente et fortement dépendante du choix de la règle de pas s_k.

2.2 Méthode de sous-gradient avec règle de Polyak

Pour pallier aux limites de la méthode précédente (sous-gradient avec un pas prédéterminé) nous introduisons cette nouvelle version de la méthode du sous gradient avec un choix de pas plus statégique.

Principe : Les itérés sont mis à jour suivant la meme formule :

$$\lambda^{k+1} = P_\Omega(\lambda^k + s_k d^k)$$

néanmoins le pas s_k est désormis choisi avec cette expression :

$$s_k = \beta_k \frac{L_k^* - L(\lambda^k)}{\|d^k\|}$$

où L_k^* est la valeur optimale du problème dual lagrangien (si connue) ou une borne supérieure sur cette dernière ou une valeur cible à atteindtre, $\beta_k \in]0, 2[$ un paramètre de relaxation. Avec cette stratégie nous ajustons dynamiquement l'amplitude du déplacement en fonction de l'écart entre la valeur courante $L(\lambda^k)$ et l'objectif visé L_k^*. Ainsi, si la solution courante est éloignée de la cible, le pas est relativement grand afin d'accélérer la progression ; au fur et à mesure qe l'on s'en rapproche, la valeur du pas diminue.

Limite : L'efficacité de cette approche dépend fortement du choix de l'objectif L_k^* puisqu'une mauvaise estimation pourrait dégrader la performance de l'approche en engendrant des pas excessif (si la valeur ciblée est surestimée) ou des pas faible (si la valeur ciblée est sous éstimée)

2.3 Méthode ADS (Average Direction Strategy)

Plutôt que de suivre exclusivement le sous-gradient courant, cette méthode définit une nouvelle direction d^k comme la bissectrice entre le sous-gradient actuel g^k et la direction à l'itération précédente d^{k-1} :

$$d_{ADS}^k = g^k + \psi_k^{ADS} d^{k-1}, o \qquad \psi_k^{ADS} = \frac{\|g^k\|}{\|d^{k-1}\|}$$

Principe : En gardant en mémoire le mouvement passé, cette approche interdit les changement brusque de la direction à chaque itération.

Limite : L'efficactité de cette stratégie est dépendant de l'historique des mouvements en mémoire. En effet, si la direction précédente était médiocre (éloignée de l'optimum), elle influencera l'itération courante.

2.4 Méthode des plans coupants

Contrairement au sous-gradient, cette méthode ne repose pas sur un déplacement itératif le long d'une direction, mais sur la construction progressive d'une approximation linéaire par morceaux de la fonction duale.

Nous résolvons à chaque étape le programme maitre (qui est une relaxation du problème dual lagrangien) :

$$(D_{Epi})^r : \begin{cases} \max & t \\ \text{s.t.} & t \leq f(x^k) + \lambda^\top g(x^k), \quad \forall k \in \{1, \dots, r\} \\ & \lambda \in \mathbb{R}_+^m \\ & t \in \mathbb{R} \end{cases}$$

Si la solution optimale (λ^r, t^r) retournée est réalisable pour le problème dual lagrangien alors elle est optimale pour ce dernier également. Si non, pour l'itération suivante, nous générons une nouvelle coupe (associée à la contrainte violée) et l'ajoutons au programme maitre.

Principe : Le concept fondamental consiste à affiner à chaque itération l'espace de recherche. Puisque chaque nouvelle coupe améliore la représentation de la fonction duale (en retranchant une partie du domaine réalisable de la relaxation). Ce processus garantit également une amélioration pour notre borne supérieure.

Limite : Le nombre de contraintes augmente à chaque itération, ce qui impacte la taille du programme linéaire. En effet, le temps de résolution de ce dernir accroit au fur et à mesure.

2.5 Tableau comparatif des méthodes

Méthode	**Points forts**	**Limites**	**Ce qui la dérange**
Sous-gradient basique	Très simple, peu coûteuse, facile à coder	Convergence lente, instable	Zigzag fréquent, dépend du pas
Sous-gradient Polyak	Pas adaptatif, souvent plus rapide, plus stable	Dépendance d'une bonne borne L_k^*	Mauvaise estimation de la cible
ADS	Réduit oscillations, plus fluide, meilleure pratique	Plus complexe, mémoire nécessaire	Ancienne mauvaise direction conservée
Plans coupants	Bonne précision, convergence solide, bornes utiles	Chaque itération coûte plus cher	Trop de coupes = modèle lourd

Table 1: Comparaison des méthodes

3 Programmation dynamique

Nous effectuerons deux relaxations pour ce problème :

$$\min \quad \sum_{i\in I}\sum_{t\in T_i} c_{it}x_{it} \tag{5}$$

$$\text{s.c.} \quad \sum_{t\in T_i} x_{it} \geq 1, \qquad \forall i \in I \tag{6}$$

$$\sum_{i\in I}\sum_{t'\in\{t,\dots,t+p_i-1\}\cap T_i} x_{it'} = 1, \quad \forall t \in \{0,1,\dots,T\} \tag{7}$$

$$x_{it} \in \{0,1\}, \qquad \forall i \in I, \forall t \in T_i \tag{8}$$

3.1 Relaxation 1 : relaxation des contraintes (6)

$\forall t \in \{0,\dots T\}$ nous posons (t) comme l'etat représentant l'instant actuel et auquel nous associons le sous problème de déterminer une séquence de pièces minimisant la somme des pénalités sur l'intervalle $\{t,\dots T\}$. Nous associons le label $L(t)$ $\forall t \in \{0,\dots T\}$ représentant le nombre minimum de pénalités à payer en remplissant l'intervalle $\{t,\dots T\}$ avec des pièces de I.

D'une part, si une pièce $i \in I$ est choisie à l'instant t, elle occupera l'intervalle $[t, t+p_i]$, générant un coût $c_{i,t+p_i}$. Il nous reste ainsi à résoudre le sous-problème de manière optimale à partir de l'instant $t+p_i$ (dont la valeur est donnée par $L(t+p_i)$). D'autre part, si $\forall i \in$: $t+p_i > T$, cet état est irréalisable, géré ainsi par $+\infty$. La relation de récurrence s'écrit comme suit :

$$\forall t \in \{0,\dots,T-1\}, \quad L(t) = \min\{\min_{i\in I, t+p_i\leq T}\{c_{i,t+p_i} + L(t+p_i)\}, +\infty\}$$

À l'instant T, l'intervalle est entièrement rempli, il n'y a plus de pénalité à payer, nous définissons donc notre état trivial avec $L(T) = 0$.

Le coût minimum pour couvrir tout l'intervalle $[0,T]$ est donné par le label optimale $L(0)$.

3.2 Relaxation 2 : relaxation surrogate des contraintes (6)

Nous obtiendrons ce PLNE :

$$\min \quad \sum_{i\in I}\sum_{t\in T_i} c_{it}x_{it} \tag{9}$$

$$\text{s.c.} \quad \sum_{t\in T_i} x_{it} = |I| \tag{10}$$

$$(7)\ (8) \tag{11}$$

$\forall t \in \{0,\dots T\}, \forall k \in \{0,\dots,|I|\}$ nous définissons notre état comme le couple (t,k) où t est l'instant actuel et k le nombre de pièces déjà traitées, dont la durée de traitement cumulée est égale à t. À chaque couple (t,k) nous associons un sous problème consisntant à minimiser les pénalités sur l'intervalle $\{t,\dots T\}$ en ordonnançant les $|I|-k$ pièces restantes. À chaque état un label $L(t,k)$ $\forall t \in \{0,\dots T\}$ est associé pour représenter le nombre minimum de pénalités à payer en remplissant l'intervalle $\{t,\dots T\}$ sachant que nous avons déjà placé k pièces pour satisfaire la contrainte (10).

Si une pièce $i \in I$ est choisie à l'instant t, elle occupera l'intervalle $[t, t+p_i]$, générant un coût $c_{i,t+p_i}$. Il nous reste ainsi à résoudre le sous-problème de manière optimale à partir de l'instant $t+p_i$ en augmentant le nombre de pièces traitées à $k+1$ (valeur stockée dans $L(t+p_i, k+1)$. Notre formule de recurrence est donnée ainsi:

$$\forall t \in \{0, \ldots, T-1\}, \forall k \in \{0, \ldots, |I|\}, \quad L(t,k) = \min\{\min_{i \in I, t+p_i \leq T} \{c_{i,t+p_i} + L(t+p_i, k+1)\}, +\infty\}$$

Notre premier cas trivial de succès survient lorsque l'horizon est rempli et toutes les pièces sont placées $k = |I|$. À ce stade aucune pénalité supplémentaire n'est présente d'où $L(T, |I|) = 0$. Les deux autres cas triviaux constituent des cas d'infaisabilité que nous gérons par ∞. D'une part, si l'horizon temporel est atteint mais il nous reste encore des pièces à traiter ($L(T,k) = +\infty$ pour $k \neq |I|$). D'autre part, si toutes les pièces ont été ordonnancées avant que l'horizon ne soit fini ($L(t, |I|) = +\infty$ pour $t < T$), l'état est invalide également.

Le coût de la solution optimale est le coût minimum nécessaire pour placer $|I|$ pièces en partant de l'instant 0 donné par $L(0,0)$ ($k = 0$, nous n'avons traité aucune pièce).

3.3 Reconstruction de la solution et vérification de sa faisabilité

Pour la relaxation 1, la reconstruction de la solution se fait en parcourant le tableaux des décisions faites (noté "choice_tab") en partant de $t = 0$. A chaque étape, nous consultons la pièce i déclarée comme étant le choix optimal à l'instant t et nous l'ajoutons à la séquence de production et nous passons de t à $t+p_i$ (fin de traitement de la pièce i). Nous répétons ce processus jusqu'a la fin de l'horizon. C'est le meme principe **pour la relaxation 2**, juste en prenant en compte non seulement l'instant t mais également k le nombre de pièces déjà traitées. En partant de l'état $(0,0)$, à chaque itération, l'indice de la pièce i avec un coût optimal est récupéré, t est mis à jour en avançant de la durée de traitement ($t = t+p_i$) tandis que le compteur de pièces augmente d'une unité ($k = k+1$). Ce mouvement assure la cohérence du chaînage jusqu'à l'épuisement total du stock de pièces à l'instant T.

Une solution est déclarée réalisable pour le problème initial si: le nombre de pièces correspond exactement à $|I|$, chaque pièces est traitée une seule fois et nous avons rempli tout l'horizon T. Dans ce cas, par dualité forte, la valeur de cette solution est optimale également. Si non, dans le cas où la solution n'est pas réalisable, la relaxation fournit simplement une borne duale pour le problème initial.

Les résultats des deux relaxations sont résumés dans le tableau (5) en notant **DB** la valeur de la borne duale retournée, **status** indiquant si la solution est réalisable pour le problème initial et **CPU** le temps d'execution.

4 Expérimentations et analyse des résultats

Afin d'évaluer les performances des méthodes implémentées, d'en effectuer une analyse comparative nous avons mener une série de tests. Ces derniers visent également à évaluer l'impact des paramètres initiaux, plus particulièrement l'influence du choix de pas pour l'algorithme du sous gradient basique. Les expérimentations ont été réalisés sur une machine (du cremi) équipée d'une carte graphique NVIDIA RTX 3060 (12 Go), d'un

processeur Intel Xeon W-1290 et de 64 Go de mémoire vive, fonctionnant sous un système d'exploitation 64 bits.

Les résultats numériques sont résumés dans le tableau présent dans l'annexe 6, en notant LB la valeur de la borne duale retournée et CPU le temps d'exécution en secondes.

La figure 1, illustre l'impact de la taille du pas sur la convergence de l'algorithme du sous gradient basique appliqué sur P_2 (1.1). Nous cherchons à atteindre la meilleure valeur duale de 0.1 au bout de 20 itérations seulement et en ajustant les multiplicateurs à chaque itération. Nous constatons que un pas élevé (la cas de $s = 2$ ou $s = 4$) entraine une instabilité initiale marquée par des oscillations représentant la chutte de la borne duale. En revanche, notre algorithme converge deux fois plus rapidement avec ces pas qu'avec un pas unitaire $s = 1$. A l'inverse un pas très faible $s = 0.25$ produit une montée régulière et quasi monotone vers l'optimum

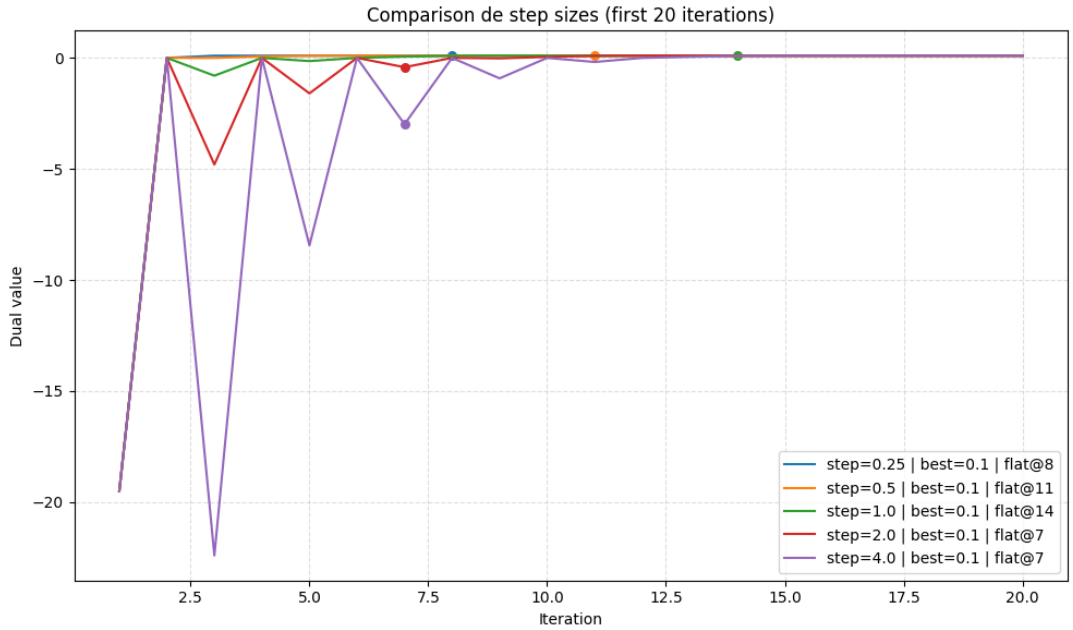

Figure 1: Comparaison des step size dans les sous-gradients basique sur P2 (initial_pi = [3, 1], max_iterations = 20, min_step_size = 1e-6)

La figure 2 présente une comaparaison entre les différents algorithmes appliqués au problème P_2 (1.1). Elle permet d'évaluer la qualité des bornes duales obtenues ainsi que l'évolution des temps de calcul de chaque méthode. Nous constatons que certaines méthodes atteignent rapidement une borne duale satisfaisante, tandis que d'autres nécessitent un nombre d'itérations important avant de se stabiliser.

La méthode du sous-gradient basique présente plusieurs oscillations. Celles-ci se traduisent par des phases successives de hausse puis de baisse de la valeur duale avant une nouvelle progression. Ce comportement s'explique par la variation des fois brutale de la direction fournie par le sous-gradient en fonction du point courant. En effet, la fonction duale n'étant pas différentiable, le sous-gradient n'est pas nécessairement unique en certains points. L'algorithme peut ainsi se déplacer d'un côté de l'optimum à une itération, puis de l'autre côté à l'itération suivante, ce qui engendre un effet de zigzag.

Par ailleurs, un comportement similaire peut être observé pour la méthode ADS. Idéalement pour un problème plus grand, les oscillations soient en général atténuées grâce

à l'utilisation de la direction précédente dans la mise à jour.

La méthode de Polyak réduit ce phénomène, car le pas dépend de l'écart entre la valeur courante et la cible. Donc lorsque l'algorithme approche de l'optimum, le pas devient plus petit, ce qui limite les dépassements et améliore la stabilité de la convergence.

Enfin, la méthode des plans coupants présente un comportement plus régulier et rapide comparant aux autres méthodes de sous-gradient. Contrairement à ces dernières, elle ne se repose pas sur des déplacements successifs selon une direction de recherche, mais sur la résolution itérative d'un programme maître enrichi progressivement par de nouvelles coupes. Chaque coupe améliore l'approximation de la fonction duale, ce qui explique la progression globalement monotone de la borne obtenue.

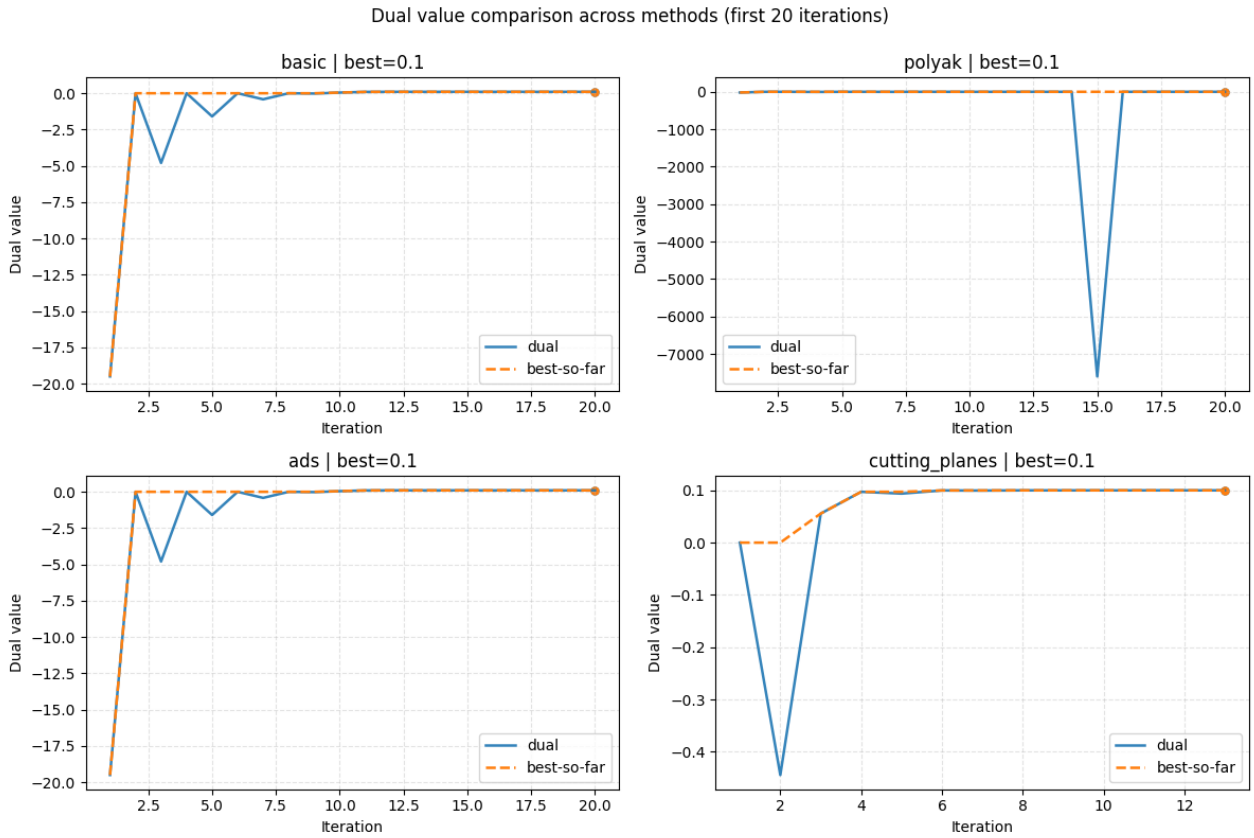

Figure 2: Comparasion des algorithmes sur P2 (initial_pi = [3, 1], max_iterations = 20, min_step_size = 1e-6)

5 Conclusion

En conclusion, la relaxation lagrangienne et la programmation dynamique sont deux approches complémentaires. La première simplifie le problème initial en fournissant de bonnes bornes, tandis que la seconde résout efficacement des sous-problèmes.

6 Annexe

SG = sous-gradient

Pb.	π_{init}	**SG-basique**		**SG-Polyak**		**SG-ADS**		**Plans coupants**	
		LB	CPU (s)	LB	CPU (s)	LB	CPU (s)	LB	CPU (s)
P1	[3, 1]	-3.0	7.36e-4	-3.0	8.34e-4	-3.0	2.18e-3	-3.0	6.70e-3
	[0, 0]	-3.0	9.37e-4	-3.14	1.12e-3	-3.0	1.31e-3	-3.0	3.55e-3
	[10, 10]	-4.0	6.05e-4	-3.01	7.71e-4	-4.0	1.21e-3	-3.0	3.06e-3
P2	[3, 1]	0.1	5.33e-4	0.1	7.15e-4	0.1	1.04e-3	0.1	1.73e-2
	[0, 0]	0.1	4.80e-4	0.1	5.17e-4	0.1	9.87e-4	0.1	1.38e-2
	[10, 10]	0.1	4.42e-4	0.1	7.31e-4	0.1	9.48e-4	0.1	1.30e-2

Table 2: Sensibilité des méthodes aux valeurs initiales de π ($\epsilon = 10^{-6}$ et $\mu_{init} = [0, 0]$)

Instance	**SG-basique**		**SG-Polyak**		**SG-ADS**		**Plan coupants**	
	LB	CPU (s)	LB	CPU (s)	LB	CPU (s)	LB	CPU (s)
wt040_001	360	261.513	1568.43	281.54	360	259.712	16672	15.014
wt040_005	360	253.337	1159.55	268.734	360	249.606	12006	15.335
wt040_025	360	430.020	20135.09	564.256	360	450.079	241293	18.312
wt050_001	450	566.517	2013.09	641.227	450	656.579	22931	36.207
wt050_005	450	558.345	1869.32	535.302	450	520.978	19952	33.658

Table 3: Comparaison des méthodes de SG sur P3 ($s_0 = 2.0$, $\alpha = 0.8$, max_iter = 100)

Instance	**SG-basique**		**SG-Polyak**		**SG-ADS**		**Plan coupants**	
	LB	CPU (s)	LB	CPU (s)	LB	CPU (s)	LB	CPU (s)
wt040_001	360	370.944	1568.43	394.166	360	409.659	16672.0	16.974
wt040_005	360	347.274	1159.55	397.237	1159.55	397.236	12006.0	14.858
wt040_025	360	423.508	20135.09	594.357	360	461.848	241293	20.0072
wt050_001	450	842.797	2013.09	923.670	450.0	881.892	22931.0	35.675
wt050_005	450	739.599	1869.32	862.806	450	746.809	19952	31.925

Table 4: Comparaison des méthodes du SG sur P4 ($s_0 = 2.0$, $\alpha = 0.8$, max_iter = 100)

dataFile	**Relaxation 1**			**Relaxation 2**		
	LB	Status	CPU (s)	LB	Status	CPU (s)
Toy_wt4.1	67	Opt	5.63e-5	67	Opt	2.26e-4
Toy_wt4.2	42	NonReal	5.04e-5	42	Opt	2.03e-4
wt040_001	396	NonReal	2.04e-2	396	NonReal	1.03e+0
wt040_002	545	NonReal	1.99e-2	545	NonReal	1.01e+0
wt040_005	766	NonReal	2.10e-2	766	NonReal	1.00e+0
wt050_001	1050	NonReal	3.36e-2	1050	NonReal	2.21e+0
wt050_002	1678	NonReal	3.15e-2	1678	NonReal	1.97e+0
wt050_005	984	NonReal	3.28e-2	984	NonReal	2.26e+0

Table 5: Comparaison des relaxations 1 et 2